

Créer le site :

Trouve ton artisan

SOMMAIRE

1. Introduction.

2. Présentation du projet.

3. Technologies utilisées.

4. Architecture du projet.

5. Maquettes Figma.

6. Base de données.

7. Sécurité.

8. Déploiement.

9. Tests.

10. Difficultés rencontrées.

11. Conclusion.

12. Liens GitHub et déploiements.

1. Introduction

Dans le cadre de cette formation en développement web, le projet **Trouve ton artisan** a été réalisé afin de concevoir une plateforme de mise en relation entre des particuliers et des artisans locaux.

L'objectif principal du projet est de permettre aux utilisateurs de rechercher facilement des artisans selon une catégorie d'activité (bâtiment, services, fabrication, alimentation), consulter leurs informations et accéder à leurs coordonnées.

Le projet a été développé avec une architecture moderne séparant :

- Un frontend React déployé sur Vercel.
- Un backend Node.js / Express déployé sur Render.
- Une base de données MySQL hébergée sur Clever Cloud.

Le projet a également été versionné avec Git et GitHub en utilisant une organisation professionnelle avec issues, branches et pull requests.

2. Présentation du projet

Le site “Trouve ton artisan” permet :

- D'afficher les catégories d'artisans ;
- De consulter les spécialités disponibles ;
- De rechercher des artisans par catégorie ;
- D'afficher les détails d'un artisan ;
- De contacter un artisan via un formulaire sécurisé.

Le site possède :

- Une page d'accueil ;
- Des pages catégories ;
- Des pages détail artisan ;
- Une page mentions légales ;
- Une gestion des erreurs 404.

Le projet suit une architecture frontend/backend découplée grâce à une API REST.

3. Technologies utilisées.

Frontend :

- React
- React Router DOM
- Axios
- Bootstrap
- Vite

Backend :

- Node.js
- Express.js
- Sequelize ORM.

Base de données :

- MySQL.

Sécurité :

- Helmet.
- CORS.
- Express Rate Limit.

Outils DevOps :

- Git.
- GitHub.
- Vercel.
- Render.
- Clever Cloud.

4. Architecture du projet.

Le projet est séparé en deux parties :

Frontend React.

Le frontend gère :

- L'interface utilisateur.
- La navigation.
- Les appels API.
- L'affichage dynamique des artisans.

Structure principale :

```
client/  
├── components/  
├── pages/  
├── services/  
├── router/  
└── assets/
```

Backend Express.

Le backend gère :

- Les routes API.
- La connexion base de données.
- Les contrôleurs.
- La sécurité.
- Les formulaires de contact.

Structure principale :

```
server/  
├── src/  
│   ├── controllers/  
│   ├── routes/  
│   ├── models/  
│   ├── middlewares/  
│   └── services/  
└──
```

Base de données.

La base MySQL contient :

- Les catégories.
- Les spécialités.
- Les artisans.
- Les messages de contact.

5. Maquettes Figma.

Les maquettes ont été réalisées avec Figma afin de préparer :

- La page d'accueil.
- Les cartes artisans.
- Les pages catégories.
- La page détail artisan.
- La navigation responsive mobile.

Les maquettes ont permis :

- D'organiser l'interface.
- De définir l'expérience utilisateur.
- D'anticiper le responsive design.

Lien Figma :

<https://www.figma.com/design/IE3yzo98u8i6pCPBzguJ3u/Trouve-ton-artisan?node-id=0-1&p=f&t=MABri2icxo7Wf7h0-0>

6. Base de données.

La base de données utilise MySQL avec Sequelize ORM.

Tables principales.

Category.

Contient les catégories d'activité :

- Bâtiment.
- Services.
- Fabrication.
- Alimentation.

Specialty.

Contient les spécialités liées aux catégories :

- Plombier.
- Electricien.
- Boulanger.
- Chocolatier.
- etc.

Artisan.

ContactMessage.

Contient les messages envoyés depuis le formulaire de contact.

Relations.

Une catégorie possède plusieurs spécialités.

`Category 1 → N Specialty`

Une spécialité possède plusieurs artisans.

`Specialty 1 → N Artisan`

ORM Sequelize.

Le projet utilise Sequelize afin de :

- Simplifier les requêtes SQL.
- Sécuriser les accès base de données.
- Gérer les relations entre tables.

7. Sécurité.

Plusieurs mécanismes de sécurité ont été mis en place.

Helmet.

Helmet sécurise les headers HTTP afin de limiter certaines attaques :

- XSS
- clickjacking
- sniffing MIME

Exemple :

```
app.use(helmet());
```

CORS.

Le backend autorise uniquement le frontend Vercel.

Exemple :

```
app.use(  
  cors({  
    origin: [  
      "http://localhost:5173",  
      "https://trouve-ton-artisan-sepia.vercel.app",  
    ],  
  }),  
);
```

Cette configuration a permis de résoudre les erreurs CORS entre Render et Vercel.

Rate Limiting.

Le formulaire de contact est protégé contre le spam.

Exemple :

```
const contactLimiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 5,  
});
```

Variables d'environnement.

Les données sensibles sont stockées dans des fichiers `.env` :

- Accès base de données.
- SMTP.
- URLs frontend/backend.

Le fichier `.env` est ignoré par Git grâce au `.gitignore`.

8. Déploiement.

Frontend.

Le frontend React a été déployé sur Vercel.

URL :

`https://trouve-ton-artisan-sepia.vercel.app`

Le frontend utilise une variable :

`VITE_API_URL=https://trouve-ton-artisan-8chh.onrender.com/api`

Backend.

Le backend Express a été déployé sur Render.

URL :

`https://trouve-ton-artisan-8chh.onrender.com`

Variables Render :

```
DB_HOST=
DB_PORT=
DB_NAME=
DB_USER=
DB_PASSWORD=
CLIENT_URL=https://trouve-ton-artisan-sepia.vercel.app
NODE_ENV=production
```

Base de données.

La base MySQL est hébergée sur Clever Cloud.

9. Tests.

Plusieurs tests ont été réalisés.

Tests API.

Route principale.

GET /

Résultat :

```
{
  "message": "API Trouve ton artisan"
}
```

Liste artisans.

GET /api/artisans

Résultat :

- Liste des artisans.

Liste catégories.

GET /api/categories

Résultat :

- Liste des catégories et spécialités.

Filtrage catégorie.

GET /api/artisans/category/plombier

Résultat :

- Récupération correcte des artisans plombiers.

Tests Frontend.

Tests réalisés :

- Navigation accueil.
- Navigation catégories.
- Détails artisans.
- Formulaire contact.
- Page 404.

Tests Console.

Les erreurs CORS ont été corrigées après configuration de :

```
CLIENT_URL
```

et du middleware :

```
cors()
```

10. Difficultés rencontrées.

Erreurs CORS.

Le frontend Vercel ne pouvait pas communiquer avec le backend Render.

Erreur :

```
blocked by CORS policy
```

Solution :

- Ajout du domaine Vercel dans `CLIENT_URL`.
- Configuration correcte du middleware `CORS`.

Gestion des slugs.

Les catégories utilisaient :

```
plomberie
```

alors que l'API utilisait :

```
plombier
```

Solution :

- Ajout d'une logique de conversion dans React.

Déploiement Render.

Le backend ne rechargeait pas automatiquement les variables d'environnement.

Solution :

- Redéploiement manuel Render ;
- Redéploiement Vercel.

11. Conclusion.

Le projet “Trouve ton artisan” a permis de mettre en pratique :

- React.
- Node.js.
- Express.
- Sequelize.
- MySQL.
- Git/GitHub.
- Déploiement cloud.

Le projet respecte une architecture moderne frontend/backend séparée et applique plusieurs bonnes pratiques de sécurité et de déploiement.

Ce projet a également permis d'apprendre :

- La gestion d'API REST ;
- Les relations base de données ;
- Les variables d'environnement ;
- La gestion des erreurs CORS ;
- Le workflow GitHub professionnel avec issues et pull requests.

12. Liens GitHub et déploiements.

Repository GitHub.

<https://github.com/chatelierjimmy-max/trouve-ton-artisan>

Frontend Vercel.

<https://trouve-ton-artisan-sepia.vercel.app>

Backend Render.

<https://trouve-ton-artisan-8chh.onrender.com>

Pulls Requests réalisés.

PR #1.

Ajout checklist de tests de déploiement.

PR #2.

Amélioration documentation de déploiement.

PR #3.

Correction gestion des slugs de spécialités.